LAYER 7 OF 13

CI/CD VERSION CONTROL

Saving Your Work and Shipping Without Breaking Things

TIER 1 — SOLOPRENEUR

MATT MURPHY .AI

mattmurphy.ai

Part of the Builder Certification Series

What This Kit Covers

This kit covers Layer 7: CI/CD & Version Control—saving your work, tracking changes, and shipping updates to your live app without breaking things. If you've ever lost code, shipped a bug to production, or couldn't figure out what changed since yesterday, this layer is for you.

You're not going to become a Git expert. You need to understand the basics: version control is how you save snapshots of your code so you can always go back. CI/CD is how you automate the process of testing and deploying your code so updates go live cleanly. Branches are working copies where you make changes without touching the live version. Commits are save points—snapshots of your code at a specific moment.

This kit walks you through three levels. Tier 1 is for solo builders who need to understand version control and use it. Tier 2 is for growing teams managing branches, pull requests, and automated pipelines. Tier 3 is for enterprise operations with complex CI/CD workflows and compliance. Start at Tier 1.

Why It Matters

Vibecoders lose work. It happens all the time. You're building with AI, making changes fast, and suddenly something breaks. You can't remember what you changed. You can't go back to the version that worked. You've lost hours—sometimes days—of progress because there's no save history.

AI tools will happily rewrite your entire codebase in one shot. Without version control, that rewrite is permanent. If it breaks something, you're stuck. Version control gives you an undo button for your entire project. CI/CD makes sure your updates actually work before they go live. Together, they're the difference between shipping with confidence and shipping with crossed fingers.

Certification Tier Map

There are three certification tiers. Each one builds on the one before it. Here's a quick look at who each tier is for and what you'll prove you can do.

Tier	Who It's For	What You Prove
Tier 1	Solo builders with a live app	You understand what version control is, how to save your work with Git, and how to deploy updates without breaking your live app.
Tier 2	Growing teams with multiple contributors	You can manage branches, pull requests, and automated CI/CD pipelines across a team.
Tier 3	Enterprise operations with compliance requirements	You can run complex release pipelines with automated testing, rollback strategies, and audit trails.

TIER 1 CERTIFICATION GOAL

You understand what version control is, why it matters, how to save your work properly using Git, and how to ship updates to your live app without breaking it.

What You Need to Know

You don't need to memorize Git commands. You need to understand the concepts so you can direct your AI tools properly and catch problems before they become disasters.

What Git actually is (save points for your code): Git is a version control system—an unlimited undo history for your entire project. Every time you "commit," you're saving a snapshot. If something breaks tomorrow, you can go back to today's snapshot. Without Git, you're building on a single copy with no safety net.

What a commit is: A commit is a save point. It captures what your code looks like right now, plus a message describing what changed. Good commits are small and frequent—like saving a document after every paragraph, not after writing the whole book.

What a repository is: A repository (repo) is where your project's code and its entire history live. Your local repo is on your computer. A remote repo (usually GitHub) is the cloud backup. "Push" sends your local save points to the cloud. "Pull" downloads the latest version.

What branches are: A branch is a working copy of your code where you can make changes without touching the live version. "Main" is your production version. Create a branch to try something new. If it works, merge it into main. If it doesn't, delete the branch—nothing is affected.

What CI/CD means: CI (Continuous Integration) automatically checks that code changes don't break anything. CD (Continuous Deployment) automatically pushes code to your live app after it passes checks. Together: code change → live on the internet, without manual steps that can go wrong.

How deployment from a repo works: Most hosting platforms (Vercel, Netlify, Railway) connect to your GitHub repo. Push code to main, they detect it, build your app, and deploy it. No FTP, no manual uploads. Push code, app updates.

Your Toolkit

At Tier 1, your hosting platform handles most CI/CD. These tools help you save your work and understand what's happening when you deploy.

GitHub (where your code lives): Your remote repository. Your code, its history, every commit—all stored here. If your laptop dies, your code survives. Also where your hosting platform pulls from.

GitHub Desktop or VS Code Git panel: Visual Git tools—commit, push, pull, and manage branches with buttons and menus instead of terminal commands. GitHub Desktop is the simplest option.

Your hosting platform's Git integration (Vercel/Netlify auto-deploy): Connect your repo once. Every push to main triggers an automatic build and deploy. This is your CI/CD pipeline at its simplest.

Deployment logs: When a deploy fails, the logs tell you why. Build errors, missing dependencies, failed deployments—check them before your users notice anything is wrong.

Certification Exam Topics

Every exam question is scenario-based. You'll see a real situation and need to identify what's right, what's wrong, or what to do next.

Version control basics: You've been building your app for two weeks with no version control. Your AI tool just overwrote a file that had working code. What's the problem, and how would version control have prevented it?

Commit practices: You've been coding all day and made dozens of changes across multiple files. You haven't committed once. What's wrong with this approach, and what should you do differently?

Repository understanding: Your code is on your laptop but not pushed to GitHub. Your laptop's hard drive fails. What have you lost, and how would a remote repository have helped?

Deployment from Git: Your app is connected to Vercel via GitHub. You push a code change to main. Walk through what happens next—how does that code change become visible to your users?

Undoing mistakes: You pushed a broken commit to main and your live app is showing errors. What are your options for fixing this without making things worse?

Branch basics: You want to add a new feature to your app but don't want to risk breaking the live version. How would you use branches to work on the feature safely?

CI/CD awareness: Your hosting platform automatically deploys every time you push to main. What's the benefit of this automation, and what's the risk if you push untested code?

Deployment logs: You pushed code and your app didn't update. Your hosting platform shows a failed build. Where do you look to find out what went wrong, and what kind of errors would you expect?

Common Pitfalls

These are the mistakes vibecoders make most often with version control and deployment. No judgment—they're easy to make. But if you recognize any of them in your own workflow, fix them before sitting for the exam.

Never committing until you're "done." If you wait until the end to save, you have no checkpoints to go back to. Commit early, commit often. Every meaningful change deserves a save point.

Writing vague commit messages like "updated stuff" or "fixed things." Your future self needs to know what changed and why. "Fixed login redirect bug" tells you something. "Updates" tells you nothing.

Editing production code directly instead of working through Git. If you change files on your server or hosting platform manually, those changes aren't tracked, can't be undone, and will be overwritten on the next deploy.

Not understanding what "push" does. Committing saves locally. Pushing sends it to the remote repo (and often triggers a deploy). These are two separate steps. If you commit but never push, your code isn't backed up.

Ignoring deployment errors and hoping they go away. When a build fails, there's a reason. The logs tell you what broke. Pushing the same code again won't fix it. Read the error.

Not having a backup of your code anywhere. If your code only exists on your laptop, you're one hardware failure away from starting over. Push to GitHub. That's your backup.

Letting AI make huge changes without committing first. AI tools can rewrite large sections of code in one go. If you don't commit before a big AI change, you can't undo it. Always save before you let AI loose.

Tier 1 Self-Assessment Checklist

Before you take the exam, run through these questions. Every "no" is something to work on.

Can you explain, in plain language, what Git does and why it matters for your project?

Do you know what a commit is and can you describe what happens when you make one?

Are you using Git (or a Git-based tool) for your current project, with regular commits?

Can you deploy your app from your repo—push to GitHub and have it go live automatically?

Do you know how to undo a mistake—either revert a commit or go back to a previous version?

Can you read your hosting platform's deployment logs and understand why a build failed?

Is your code in a hosted repository (GitHub) so it's backed up and accessible from anywhere?

AI Audit Prompt Template

Copy this prompt into your AI coding tool to get a quick health check on your version control and deployment setup. It checks the same things the certification exam covers.

```
Review my app's version control and CI/CD setup and check the following. For each one, tell me pass or fail with a specific example:

Commit frequency: Am I committing regularly, or are there long gaps between save points where I could lose work?

Commit messages: Are my commit messages descriptive enough that I could understand what changed three months from now?

Branch usage: Am I working directly on main, or using branches to keep my live app safe while making changes?

Deployment setup: Is my repo connected to my hosting platform so deploys happen automatically when I push?

Backup status: Is my code pushed to a remote repository, or does it only exist locally on my machine?

Deployment logs: Have I checked my hosting platform's build logs recently? Are there any failed builds or warnings I'm ignoring?

Give me an overall score out of 6 and list the top 3 things to fix first.
```

What's Next

Once you've gone through this kit and can answer "yes" to the self-assessment checklist, you're ready for the Layer 7 certification exam at your target tier.

The best way to prepare: go look at your actual Git history and deployment setup. Open GitHub and review your recent commits—are the messages clear? Check your hosting platform's deploy logs—are builds succeeding? Try reverting a commit to see what happens. Ask your AI tool to explain your deployment pipeline. The exam tests whether you understand your own setup, and that understanding comes from looking at real projects.

CERTIFICATION PATHWAY

Associate Builder: Pass all 13 layers at Tier 1

Certified Builder: Pass all 13 layers at Tier 1 + Tier 2

MADE Certified: Pass all 39 tier exams + capstone project

Each exam requires 80% to pass. You can retake after a 24-hour cooldown. No rush—take the time to build something real first.

MATT MURPHY .AI

mattmurphy.ai